BhookBuster

AI Integration Plan

Semantic Search · Recommendations · Business Analytics

This document covers the full plan for adding three AI capabilities to the BhookBuster food-delivery platform: natural-language semantic food search, a personalised recommendation engine, and an AI-powered business analyst dashboard for sellers and admins. It also documents the security foundation that must be in place before any AI feature ships.

4 / 10	3 criticals	8 steps	3 features
Overall health (pre-AI audit)	Security issues to fix first	Recommended build order	AI capabilities planned

01 — Audit scorecard

A full codebase audit was run using Codex before designing the AI features. The table below shows current health scores. Every AI feature is gated behind resolving the three critical security findings.

Area	Score	Status
Security	3 / 10	Critical
Performance	5 / 10	Needs work
Maintainability	4 / 10	Needs work
Test coverage	0 / 10	Critical
Architecture quality	5 / 10	Acceptable
Overall health	4 / 10	

02 — Security foundation (must ship first)

No AI feature should be deployed until these are resolved. Building ML/AI capabilities on top of insecure surfaces amplifies the blast radius of each vulnerability.

! **Browser-exposed internal key**
VITE_INTERNAL_SERVICE_KEY is readable by any user in browser devtools. Replace direct internal realtime emit with an authenticated backend API endpoint.

! **Unauthenticated payment and upload routes**
Payment verification and Cloudinary upload endpoints have no auth middleware. Add isAuth + ownership checks; enforce file type/size limits and rate limiting.

! **Open CORS across all services**
Most services and Socket.IO use origin: '*'. Pin allowed origins to known frontend domains via environment config per deployment.

! **Broken restaurant ownership checks**
Any seller can fetch any restaurant's paid orders. Fix by joining restaurantId with ownerId === req.user._id before any data is returned.

! **Secrets tracked in git**
services/auth/.env is committed to the repository. Rotate all credentials, remove the file from history, and commit only .env.example going forward.

! **Zero test coverage**

No unit, integration, or e2e tests exist. Add tests for auth, ownership, payment idempotency, and analytics scoping before the AI features add new endpoints.

03 — AI-powered semantic food search

3 / 5
Readiness score

Replace the current regex name search with natural-language understanding. Customers can search 'healthy dinner', 'spicy chicken under Rs.300', or 'something light after a workout' and receive results ranked by meaning.

Integration points

What	File / model	Detail
Embed menu items	*services/restaurant — MenuItems.ts*	Embed name, description, cuisine, tags, price, dietary flags, spice level.
Embed restaurants	*services/restaurant — Restaurant.ts*	Embed name, description, location label, cuisine, verification status.
Replace regex search	*services/restaurant — restaurant.ts line 207*	Augment $regex with hybrid vector + metadata filter endpoint.

Phased rollout

MVP Week 1-2	Add metadata fields, embedding pipeline, authenticated POST /api/search/semantic, Home UI result cards.
v1 Month 1	Query parser for price/dietary constraints, popularity ranking blend, search analytics events.
v2 Quarter 1	Multi-modal image/menu understanding, LLM re-ranker, personalized semantic search.

Prerequisite security fixes

- Remove VITE_INTERNAL_SERVICE_KEY from browser
- Lock CORS and service URLs
- Add test harness before changing search behaviour

Technology stack: MongoDB Atlas Vector Search + OpenAI text-embedding-3-small

04 — Personalized food recommendation system

2 / 5
Readiness score

Recommend dishes and restaurants from a user's order history, favourite cuisines, past searches, and preferences. New users see popular nearby options; returning users see a personalised 'For You' feed.

Integration points

What	File / model	Detail
Order history signals	*services/restaurant — Order.ts line 45*	Paid orders, items, subtotal, restaurant, timestamps.
Cart intent signals	*services/restaurant — Cart.ts line 12*	Current cart and abandoned-intent events.
Event-driven profile updates	*services/restaurant — payment.consumer.ts line 10*	Trigger recommendation profile refresh on PAYMENT_SUCCESS.

Phased rollout

MVP Week 1-2	Popular nearby fallback, UserTasteProfile schema, content-based 'For You' row on home feed.
v1 Month 1	Capture search/click/favourite/rating events, RabbitMQ profile updater, explainable recommendations.
v2 Quarter 1	Collaborative filtering, promo-aware recommendations, LLM re-rank with explanations.

Prerequisite security fixes

- Fix unauthenticated payment routes before using orders as training truth
- Rate-limit and validate all event capture endpoints
- Ensure PAYMENT_SUCCESS events are trustworthy

Technology stack: Embedding similarity (reuse search vectors) + RabbitMQ consumer + UserTasteProfile collection

05 — AI business analyst dashboard

3 / 5
Readiness score

Give admins and restaurant owners AI-generated narrative insights, anomaly flags, and business recommendations beyond the current Recharts charts — plus a natural-language 'ask your data' interface scoped by role.

Integration points

What	File / model	Detail
Scoped analytics endpoint	*services/restaurant — order.ts line 194*	Replace unsafe seller order fetch with owner-scoped aggregation endpoints.
Seller dashboard	*frontend — Restaurant.tsx line 88*	Replace placeholder Sales Page with RestaurantInsights component.
Admin dashboard	*frontend — Admin.tsx line 50*	Add PlatformInsights panel and chat-style analytics box.

Phased rollout

MVP Week 1-2	Secure owner-scoped aggregate endpoints, top dishes/revenue/peak hours, static insight cards.
v1 Month 1	LLM narrative summaries, anomaly detection, admin platform-wide view.
v2 Quarter 1	Ask-your-data templates, demand forecasting, campaign recommendations.

Prerequisite security fixes

- Fix restaurant ownership checks before any analytics
- Strip PII fields before LLM sees them (deliveryAddress.mobile)
- Add audit logging for every AI analytics query

Technology stack: Mongo aggregation pipelines + LLM narrative layer + pre-approved NL query templates

06 — Shared AI infrastructure

All three AI features share a single infrastructure layer. Building this once avoids duplicated model calls, scattered API keys, and inconsistent rate limiting.

Centralizes all model API calls, keys, rate limits, retries, and prompt ver
/internal/rerank, /internal/insights. Never reachable from the frontend —
the current shared x-internal-key.

Menu items, restaurants, and user search queries all go through a singl
OpenAI text-embedding-3-small. Embeddings are regenerated on creat
controllers.

Vectors live in the same MongoDB instance already used by Mongoose
(price, dietary, geo radius) so hybrid search requires no additional datas

A new USER_EVENT_QUEUE carries search, click, cart, favourite, rati
recommendations consumer listens and updates UserTasteProfile — ri
introducing new infrastructure.

07 — Recommended build order

Follow this sequence. Each step unblocks the next. Do not start an AI feature until all steps above it are merged and tested.

1	**Security foundation** Fix CORS, upload/payment auth, remove frontend internal keys, correct owner-scoped order analytics, set up test harness.
2	**services/ai-gateway** Scaffold new service. Internal model calls, HMAC auth, rate limits, secrets management, structured logging.
3	**Menu + restaurant metadata and embedding pipeline** Add new schema fields (cuisine, tags, dietary flags, spice level). Build embedding generation and storage.
4	**Semantic food search MVP** Authenticated POST /api/search/semantic. Hybrid vector + metadata filter. Home UI result cards.
5	**User event stream + recommendation profile consumer** USER_EVENT_QUEUE on RabbitMQ. UserTasteProfile collection. Consumer updates profile on order and event.
6	**Personalized recommendations MVP** GET /api/recommendations/home. Content-based using taste profile + search embeddings. For You row in frontend.
7	**Scoped analytics aggregation endpoints** Owner-scoped Mongo aggregation endpoints. Revenue, peak hours, top dishes, repeat-customer rate.
8	**AI business analyst dashboard** LLM narrative summaries, anomaly detection, ask-your-data templates, insights panels in admin/seller dashboard.

After Codex produces the design document for each feature, run a targeted follow-up: 'Now implement feature N, MVP scope, with integration tests.' This keeps implementation grounded in the file-level analysis rather than generic boilerplate.